

Data Architecture Reference

A measured reference for data architecture and state design

Edition 1 (draft)

Built from commit repo/book-typst-toolchain-v1-4-gbcec82f (bcec82ff83747455e4a8bf6fcca02adb85733dc8)
— working tree true.n Typst 0.15.1 (unknown commit) (image
sha256:032e292249bcd378480cc7c142cfa324b63ef8aadeb88d7e7230320c4c9c422f).n Evidence registry book/evidence/v2/
claims.json, digest 976e3635f5d2f769c3ab71e35e13083dc13bc728a82be32e10a7bfbe5f12e756.

1 Edition and provenance

This is a generated artefact: Typst source plus the evidence registry, compiled in a pinned Podman image. **Every claim in this book must resolve to a claim in the v2 registry**, and every number must resolve to a measured cell in a cited report. Nothing here is hand-entered at build time; the fields below are injected from the build script.

Source commit	bcec82ff83747455e4a8bf6fcca02adb85733dc8
Source describe	repo/book-typst-toolchain-v1-4-gbcec82f
Working tree at build	true
Built at (UTC)	2026-09-22T19:24:48Z
Typst version	0.15.1 (unknown commit)
Typst image digest	sha256:032e292249bcd378480cc7c142cfa324b63ef8aadeb88d7e7230320c4c9c422f
Evidence registry	book/evidence/v2/claims.json
Evidence digest	976e3635f5d2f769c3ab71e35e13083dc13bc728a82be32e10a7bfbe5f12e756
Source tree hash	a0b4944749232d7dbe695017f8105f8730de0b613a698aecb78bb4f5a41ac646

The book is assembled with Typst inside a pinned Podman image; the evidence index at the end is rendered from the registry file at build time.

Coverage gap

Environment and provenance limits travel with every number: all results come from one host (host-zenbook-ux5406sa) with shared cores and no real network between “cluster” nodes. Run tags mark the **producing code state**, not a tree containing the results. A reader reproducing a run must check out the run tag to get the code, then read the committed results directory.

2 How to read a number here

- Every family, variant, topology and scenario section follows the same progressive structure: quick choice, where it thrives and perishes, common scenarios, mechanism, evidence and reproduction.
- A **direct** callout is measured in the cited run. An **analogy** callout is a labelled transfer to a family that was **not** measured. A **gap** callout is missing evidence.
- Gaps are never null results: “not measured” must not be read as “would not change this”.
- No cross-study numeric ranking appears anywhere; no comparability record exists.

Contents

1	Edition and provenance	3
2	How to read a number here	4
3	How to choose	7
3.1	Step 0 — answerability before performance	7
3.2	Step 1 — place each fact once	7
3.3	Step 2 — name the contended surface	7
3.4	Step 3 — decide who maintains derived state	7
3.5	Step 4 — state the evidence level and the gap	7
3.6	A worked decision	8
4	Major families	9
4.1	Normalized facts	9
4.2	Rolldown / duplicated data	10
4.3	Rollups / materialized aggregates	10
4.4	Embedded documents	11
4.5	Append-only history / snapshots	12
4.6	External read copies / caches	13
5	Minor variants	15
5.1	Measured controlled pairs	15
5.2	Named but not measured as controlled pairs	16
6	Cross-family comparisons	17
6.1	What a controlled pair looks like	17
6.2	Comparisons this book deliberately does not make	17
6.3	Direct evidence for the two family contrasts	17
6.4	The confounds, published here rather than in a footnote	17
7	Topologies	18
7.1	What was measured	18
7.2	What topology does to a design	18
7.3	Open topology gaps	18
7.4	Boundaries	18
8	Scenarios	19
8.1	Ticket overbooking and refunds	19
8.2	Reserved seating and holds	19
8.3	Configuration portal	20
8.4	Donor portal with an external cache	20
8.5	Scenario transfers are labelled	21
9	Evidence and reproduction	22
9.1	The provenance model	22
9.2	Reproducing the book itself	22
9.3	Reproducing a measured run	22
10	Concepts	23
11	Information placement	24
11.1	The ladder	24
11.2	The rule that comes before performance	24
11.3	Boundaries	24
12	Indexes and hot rows	25
12.1	Indexes	25

12.2	Hot rows	25
12.3	Direct evidence	25
12.4	Boundaries	25
13	Concurrency control	26
13.1	Optimistic (version-checked)	26
13.2	Pessimistic (lock-before-update)	26
13.3	The negative control matters here	26
13.4	Boundaries	26
14	Derived state and history	27
14.1	Maintenance is the design	27
14.2	History is answerability	27
14.3	Direct evidence	27
14.4	Boundaries	27
15	Expiry and clock authority	28
15.1	What the corpus shows	28
15.2	What it cannot yet show	28
15.3	Boundaries	28
16	Cache consistency	29
16.1	The gain	29
16.2	The failure mode	29
16.3	Freshness is a contract, not a speed knob	29
16.4	Boundaries	30
17	Distributed placement	31
17.1	What the corpus measures	31
17.2	What it does not measure	31
17.3	Boundaries	31
18	Reading benchmarks honestly	32
18.1	Six questions to ask of any number	32
18.2	Worked examples of a number that cannot be quoted	32
18.3	Boundaries	32
19	Evidence registry (active v2 claims)	33
19.1	normalized facts	33
19.2	rolldown / duplicated data	34
19.3	rollups / materialized aggregates	34
19.4	embedded documents	35
19.5	append-only history / snapshots	36
19.6	external read copies / caches	37
19.7	Retired v1 claims	37

3 How to choose

CHAPTER · HOW TO CHOOSE

This chapter is the decision path. It front-loads the two questions that dominate every later trade-off: what must the schema be able to answer, and who arbitrates the contended row.

3.1 Step 0 — answerability before performance

Direct evidence

Write down the questions the data must answer **after** an undo, a cancellation or a correction. Two measured answerability failures make this a rule rather than a preference: refunds were unanswerable in 14 of 15 ticketing designs, and no reserved-seating design could size a hold window because an expired hold is erased.

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33-35% more storage.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: answerability is a schema fact, not a timing; the +33-35% storage delta is deterministic and stable across three runs; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

If a required question is unanswerable, no index or cache fixes it. Choose a family that can answer it — usually append-only history — and accept its maintenance cost.

3.2 Step 1 — place each fact once

Walk the derive → index → materialize → copy/cache ladder (see the **information placement** concept) for each question. Prefer the lowest rung that meets the requirement, because each rung up adds maintenance, storage or a consistency contract.

3.3 Step 2 — name the contended surface

Find the rows many writers touch at once. If there is one, choose an arbitration strategy explicitly: pre-created units with compare-and-set, SKIP LOCKED, or a version-checked update. The counter-row design was 4–8x slower than pre-created seats and blocked unrelated edits.

3.4 Step 3 — decide who maintains derived state

For every rollup or copy, name the maintainer and the failure it must never have. A trigger makes the guarantee structural; application maintenance keeps writes cheap but must be atomic with its publication. The measured trigger cost was 6.85x on whole-configuration replacement.

3.5 Step 4 — state the evidence level and the gap

Every recommendation carries a claim id, its strength, its confounds and its coverage gap. A recommendation without a gap statement is over-claiming.

3.6 A worked decision

- A ticketing system must never oversell and must report refunds → append-only ledger plus a pre-created-seat arbiter, +33–35% storage.
- A configuration portal reads its overview far more than it writes → application-maintained rollup; a trigger only if a forgotten write is unacceptable.
- A public donor portal serves warm profiles → cache with a stated freshness contract and a fence, accepting that the cache's budget is outside the database comparison.

4 Major families

CHAPTER · MAJOR FAMILIES

A family is a way of deciding where a fact lives. The six here are fixed by the goal; each section follows the same progressive structure, and the mechanism chapters carry the reusable explanations. Every cost below is a measured claim, labelled **direct** when it comes from the cited run or **analogy** when it is a transfer.

4.1 Normalized facts

4.1.1 Quick choice in plain language

Keep each fact in one place and let queries assemble the answer. Choose it unless a measured question says otherwise.

4.1.2 Where it thrives and where it perishes

Thrives when questions are answerable from the rows themselves and writes are spread across many parents. Perishes on a hot row, or when the required question was never modelled.

4.1.3 Common scenarios

Order lines, ticket sales, configuration entries — anything that must never overbook or double-sell.

4.1.4 Mechanism

See the **indexes and hot rows** concept and the **concurrency control** concept. Indexes answer expressible questions; arbitration answers contended rows.

4.1.5 Costs

Reads pay per query; writes are cheap; storage is the baseline; correctness depends on the constraint you actually declared; operations are ordinary.

4.1.6 Direct evidence

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-01-study01-d2d3-copied-key-package](#)

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4-8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: [studies/02-ticket-booking/reports/20260913T021206Z.md](#) · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

4.1.7 Boundaries and reproduction

The ratios are single-run and single-host. Reproduce from the run tags in the cards; check the `run_level` note for failed cells before quoting a percentage.

4.2 Rolldown / duplicated data

4.2.1 Quick choice in plain language

Copy a parent value onto the child to remove a join. Do it when the copied value is small, stable enough, and the join is on the hot path.

4.2.2 Where it thrives and where it perishes

Thrives on read-heavy workloads with a bounded parent set. Perishes when the copied value changes often, because every change becomes a fan-out write.

4.2.3 Common scenarios

A donation row carrying its donor's display name; an order line carrying its product's category.

4.2.4 Mechanism

See the **information placement** concept. The measured pair is a package — the copied key plus its SQL and indexes — so attribute the gain to the package, not to the column alone.

4.2.5 Costs

Reads improve; writes gain an index and a copy to maintain; storage grows; correctness needs a rule for the stale copy; operations must backfill after a schema change.

4.2.6 Direct evidence

v2-05-d3-read-advantage-repeated

rolldown / duplicated data · repeated_controlled

D3's (`d3_flattened_fk`) read advantage over the normalized indexed design survives five fresh loads; the recency SQL and actual heap avoidance explain the gain, while its insert cost remains imprecise.

Trials: 1 whole-run replications, 5 fresh-load trials per cell, 1 inner iterations.

Limits: five fresh-load trials per cell, one session; the insert-cost estimate is imprecise and must not be quoted as a ratio

Source: studies/01-charity-tree/reports/20260913T125342Z-v3.md · run 20260913T125342Z-v3 · tag run/01-charity-tree/20260913T125342Z-v3

4.2.7 Boundaries and reproduction

One family contrast is not a general law. The copied-key package is confounded (`conf-01`), and placement is unverified (`v2-gap-05`).

4.3 Rollups / materialized aggregates

4.3.1 Quick choice in plain language

Store the total on the parent. Use it when an aggregate is read far more often than its children change, and be explicit about who maintains it.

4.3.2 Where it thrives and where it perishes

Thrives on dashboards and parent-level reads. Perishes when child writes dominate, or when a write path can forget the rollup.

4.3.3 Common scenarios

Donor lifetime totals, event ticket counts, a configuration portal's overview.

4.3.4 Mechanism

See the **derived state and history** concept. Trigger-maintained is a structural guarantee at a write cost; application-maintained is cheaper but must be atomic with the publication.

4.3.5 Costs

Reads win big; writes pay maintenance; storage grows modestly; correctness is the whole game; operations must rebuild the rollup after a bulk load.

4.3.6 Direct evidence

v2-02-rollup-family

rollups / materialized aggregates · single_run_directional

Parent rollups win the aggregate read score (D4 rollup/trg 73.8x and D5 rollup/app 69.9x against D1 minimal) and pay for it on inserts (2.6k/s and 2.1k/s against 6.2k/s).

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; PostgreSQL 1 node numbers shown; YugabyteDB in the same report; the write cost is per-insert on a fresh load

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-13-trigger-rollup-cost

rollups / materialized aggregates · single_run_directional

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (n4_rollup_app) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-04-recency-maintained-index

rollups / materialized aggregates · single_run_directional

The recency flag works, but an index on the rollup column answers the same question faster: comparing d20_recency_flag with d22_recency_rollup_idx in the same run gives the reported read speedup on PostgreSQL and YugabyteDB.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 1 inner iterations.

Confounds: conf-02-study01-d20d22-write-axis

Limits: the write comparison is confounded: d22 also maintains D4's multi-column rollup package, so its write cost is not the index alone; three trials per cell; claim-02's v1 trials field was recorded as 1 and is corrected here

Source: studies/01-charity-tree/reports/20260916T090036Z-v3.md · run 20260916T090036Z-v3 · tag run/01-charity-tree/20260916T090036Z-v3

4.3.7 Boundaries and reproduction

The recency pair's write cost is confounded (conf-02); quote its read result, not its write cost.

4.4 Embedded documents

4.4.1 Quick choice in plain language

Put the children inside the parent when they are read together and bounded in size. It is a read-locality choice, not a database-family choice.

4.4.2 Where it thrives and where it perishes

Thrives when a bounded child set is always read with its parent. Perishes when children are queried independently or grow without bound.

4.4.3 Common scenarios

A reservation's seat map, a configuration file rendered whole.

4.4.4 Mechanism

See the **information placement** concept. A section document answered a 100k-seat map 4.5–11x faster than per-seat rows on YugabyteDB; a sharded variant collapsed point lookups to 26 ops/s.

4.4.5 Costs

Reads of the whole child set improve; point access to one child can get much worse; storage is similar; correctness needs a bounded-growth rule; operations can be awkward for partial updates.

4.4.6 Direct evidence

v2-03-embedding-not-ahead

embedded documents · single_run_directional

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: 2.4x D1 minimal's read score, below both indexed designs, with a 0.57x insert rate.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5-11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: conf-03-study03-layout-family-package

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

4.4.7 Boundaries and reproduction

Embedding here means a JSON/array child column in PostgreSQL or a document row in YugabyteDB — a labelled **analogy** to a native document database, which was not measured (v2-gap-06). The row-versus-document contrast is a package comparison (conf-03).

4.5 Append-only history / snapshots

4.5.1 Quick choice in plain language

Never overwrite the fact; append what happened. Choose it when a question must survive an undo.

4.5.2 Where it thrives and where it perishes

Thrives wherever refunds, cancellations or corrections are in scope. Perishes under storage limits with no retention policy.

4.5.3 Common scenarios

Refund reporting, audit trails, last-purchase questions after a cancellation.

4.5.4 Mechanism

See the **derived state and history** concept and the **expiry and clock authority** concept.

4.5.5 Costs

Reads need a report over history; writes append and fence; storage grows (about 33–35% more in the ticketing ledger); correctness **improves** — this family is the answerability fix; operations need retention.

4.5.6 Direct evidence

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33-35% more storage.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: answerability is a schema fact, not a timing; the +33-35% storage delta is deterministic and stable across three runs; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-08-ledger-race-cost-128-buyers

append-only history / snapshots · single_run_directional

At 128 buyers the ledger costs 1.1-2.3x of the sell-out race on PostgreSQL and 1.8-2.2x on YugabyteDB with tight spreads, 2.1-2.5x on the refund itself, and not one under-booked seat anywhere; it delays the organiser's own edit by up to 5x.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 128 inner iterations.

Confounds: conf-05-study02-buyer-arms-disagree

Limits: one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier

Source: studies/02-ticket-booking/reports/20260921T205212Z.md · run 20260921T205212Z · tag run/02-ticket-booking/20260921T205212Z

4.5.7 Boundaries and reproduction

The PostgreSQL ledger cost is order-sensitive and unsettled (v2-09); the load multiplier is not published (v2-gap-01). Quote the 128-buyer scoped result, never a pooled ratio.

4.6 External read copies / caches

4.6.1 Quick choice in plain language

Serve the answer from beside the database, with a stated freshness contract. Do it for warm, cacheable, read-heavy questions.

4.6.2 Where it thrives and where it perishes

Thrives on repeated reads of stable keys. Perishes when many instances share a cache under relaxed freshness without a fence.

4.6.3 Common scenarios

A donor portal's hot profile reads, a public dashboard over changing totals.

4.6.4 Mechanism

See the **cache consistency** concept. Invalidation is a fence that refuses a stale publication, not a deletion.

4.6.5 Costs

Reads gain 2.2–3.5x and p99 drops an order of magnitude; writes may gain a fence; storage moves to the cache (whose budget is outside the comparison); correctness needs the fence; operations gain a new failure domain.

4.6.6 Direct evidence

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2-3.5x in throughput and an order of magnitude in p99 (15.8-27.1 ms to 0.8-2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); no YugabyteDB, cluster, verified placement or open-loop demand

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: `conf-06-claim12-full-noise-range`, `conf-04-study05-db-only-cache`

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: `studies/05-cache-consistency/reports/20260921T-survey3.md` · `run 20260921T-survey3` · `tag run/05-cache-consistency/20260921T-survey3`

4.6.7 Boundaries and reproduction

Redis was measured as a cache, never as an authoritative store; the result is `db-only` framed (`conf-04`). The three-instance failure is a mechanism demonstration, not a rate to predict from.

5 Minor variants

CHAPTER · MINOR VARIANTS

A minor variant is one controlled change inside a family. The corpus has measured pairs for some; the rest are named with their evidence status so a reader can tell the difference.

5.1 Measured controlled pairs

5.1.1 Index selection

Adding the query index was the first large read win in the charity tree (D2 indexed, 13.3x the minimal design's read score); flattening the foreign key reached 20.5x.

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-01-study01-d2d3-copied-key-package](#)

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · run 20260912-small · tag none (legacy)

5.1.2 Optimistic versus pessimistic concurrency

Under one hot key and 16 writers, lock-before-update delivered 1.76x the version-checked design — an upper bound, because the run could not separate the lock's advantage from extra per-retry work. An unchecked read-modify-write lost 93.7% of acknowledged updates.

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: [studies/04-configuration-portal/reports/20260921T1215Z-small.md](#) · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

5.1.3 Trigger versus application maintenance

Trigger-maintained rollups cost 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 to 42 ms); the application rollup kept writes near the reference for the same read gain.

v2-13-trigger-rollup-cost

rollups / materialized aggregates · single_run_directional

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (n4_rollup_app) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: [studies/04-configuration-portal/reports/20260921T1215Z-small.md](#) · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

5.1.4 Strict versus relaxed freshness

Every controlled strict/relaxed pair differed by -14.2% to $+12.3\%$ in read throughput — inside the noise. The visible cost of strictness is on the write path.

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and $+12.3\%$, inside the run's $\sim 20\%$ single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-06-claim12-full-noise-range](#), [conf-04-study05-db-only-cache](#)

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

5.2 Named but not measured as controlled pairs

- **Full versus bounded embedding.** Embedding was measured as a family, not against a bounded variant.
- **Locks / CAS / isolation levels.** The corpus has CAS and lock designs, but not an isolation-level sweep under one invariant.
- **Pre-created versus on-demand inventory.** Measured in ticketing as a family contrast with arbitration, not isolated from concurrency strategy.
- **Cache-aside versus write-through.** Both appear; the three-instance failure is about fencing, not about the two patterns head-to-head.

Coverage gap

No variant above is a cross-study ranking, and none is an equal-total-resource comparison. Treat the unmeasured variants as open, not as null results.

6 Cross-family comparisons

CHAPTER · CROSS-FAMILY COMPARISONS

Comparisons across families are where over-claiming happens. The rule here: a pair is quoted only when one decision changed, and every comparison carries its confounds on the page.

6.1 What a controlled pair looks like

- **One decision changed.** The charity tree's index and rolldown pairs each change one placement decision; their confounds are named (conf-01).
- **Same workload and resources.** Logical data, operations and per-node budget held constant.
- **Controls present and fired.** A pair whose negative control did not fire is weaker than it looks.

6.2 Comparisons this book deliberately does not make

- **No cross-study numeric ranking.** No comparability record exists, so ratios from different runs are not pooled.
- **No equal-total-resource ranking.** Single-versus-cluster answers “add machines”, not “same budget”.
- **No native-family transfer.** JSONB/state is not a native document store and Redis was measured only as a cache; such a transfer is analogy, labelled as such.

6.3 Direct evidence for the two family contrasts

v2-03-embedding-not-ahead

embedded documents · single_run_directional

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: 2.4x D1 minimal's read score, below both indexed designs, with a 0.57x insert rate.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5-11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-03-study03-layout-family-package](#)

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

6.4 The confounds, published here rather than in a footnote

- **conf-01** — the D2→D3 rolldown prices a copied key plus its SQL and indexes, not the column.
- **conf-02** — the recency pair's write axis is confounded by the whole rollup package.
- **conf-03** — the row-versus-document seat map is a family/package comparison.
- **conf-04** — the cache gain is db-only framed.
- **conf-05** — the ticketing buyer arms disagree in sign on PostgreSQL.
- **conf-06** — the strict-freshness range must be shown in full (−14.2% to +12.3%).

7 Topologies

CHAPTER · TOPOLOGIES

Topology is a separate axis from design family: node count, replication, placement, endpoint routing, resource framing, network conditions and failure behaviour. A design that is best on one node is not thereby best on three.

7.1 What was measured

Studies 01–03 run PostgreSQL single-node and YugabyteDB single-node (RF=1) and 3-node (RF=3). Every cell is throttled to a documented per-node budget, and every manifest records it. The cost model is therefore “per-node budget”, not “equal total budget”.

7.2 What topology does to a design

Replication turns a write into a consensus write, so write-heavy designs move differently from read-heavy ones. The section-sharded seat map collapsed point lookups to 26 ops/s on YugabyteDB, while the same family’s whole-map read was 4.5–11x a row design — one topology change, opposite directions for two access patterns.

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5–11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-03-study03-layout-family-package](#)

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: [studies/03-reserved-seating/reports/20260921T231328Z.md](#) · run 20260921T231328Z · tag [run/03-reserved-seating/20260921T231328Z](#)

7.3 Open topology gaps

Coverage gap

Physical colocation is unverified (v2-gap-05); balanced endpoint routing is unproven (v2-gap-07); there is no real network between “cluster” nodes. Multi-node results here are placement experiments, not distributed-systems results.

7.4 Boundaries

Do not scale these numbers to production capacity. The nodes share one host’s eight cores, are CPU-throttled, and communicate over a loopback-class path.

8 Scenarios

CHAPTER · SCENARIOS

A scenario is a set of questions plus a workload shape. The same family can be right in one and wrong in another, so each scenario below states its answerability question and its evidence status.

8.1 Ticket overbooking and refunds

Never oversell, then report refunds. Pre-created seats with compare-and-set or SKIP LOCKED sold 4–8x faster than a counter row; a ledger made refunds answerable at +33–35% storage. The ratio between the two arbitration designs at 128 buyers is 1.1–2.3x on PostgreSQL; the small arm is unsettled.

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4-8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: studies/02-ticket-booking/reports/20260913T021206Z.md · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-08-ledger-race-cost-128-buyers

append-only history / snapshots · single_run_directional

At 128 buyers the ledger costs 1.1-2.3x of the sell-out race on PostgreSQL and 1.8-2.2x on YugabyteDB with tight spreads, 2.1-2.5x on the refund itself, and not one under-booked seat anywhere; it delays the organiser's own edit by up to 5x.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 128 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier

Source: studies/02-ticket-booking/reports/20260921T205212Z.md · run 20260921T205212Z · tag run/02-ticket-booking/20260921T205212Z

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92-283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7-5.7x (not the headline's narrower 1.7-2.1x).

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 32 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

8.2 Reserved seating and holds

Hold, expire, confirm, and re-sell. Guarantees held everywhere, but YugabyteDB refused valid confirmations until the refused statement was retried; and the funnel that would size the hold window is unanswerable because releasing an expired hold erases the evidence.

v2-10-guarded-confirm-refusals

normalized facts · single_run_directional

On YugabyteDB a guarded confirmation is not enough: correct designs refused valid holds, and retrying the refused statement once removed every refusal at no measurable cost; on PostgreSQL the guarantee held everywhere.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag

Source: studies/03-reserved-seating/reports/20260915T002411Z.md · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

8.3 Configuration portal

Read the overview often, replace configurations occasionally. A trigger-maintained rollup cost 6.85x on replacement; the application rollup kept writes near the reference. This is a small-scale result at one cardinality.

v2-13-trigger-rollup-cost

rollups / materialized aggregates · single_run_directional

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (n4_rollup_app) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

8.4 Donor portal with an external cache

Serve warm profile reads from a cache. Throughput rose 2.2–3.5x and p99 fell an order of magnitude; three instances sharing one relaxed cache recorded about 87% wrong reads unless the publication was fenced.

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2-3.5x in throughput and an order of magnitude in p99 (15.8-27.1 ms to 0.8-2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: conf-04-study05-db-only-cache

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); no YugabyteDB, cluster, verified placement or open-loop demand

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: `conf-04-study05-db-only-cache`

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: `studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3`

8.5 Scenario transfers are labelled

Analogy — not measured here

A supermarket stock counter, an auction bid ledger or an e-commerce inventory pool is structurally similar to the ticketing hot row, but none of them was measured here. Use those transfers as analogies from the hot-row mechanism, never as separate evidence.

9 Evidence and reproduction

CHAPTER · EVIDENCE AND REPRODUCTION

Every number in this book resolves to a claim in `book/evidence/v2/claims.json`, and every claim to a measured cell in a cited report. This chapter explains the confidence cards, the run-tag provenance model, and how to reproduce a run in Podman.

9.1 The provenance model

Direct evidence

A run tag marks the producing code state, not a tree that contains the run's results. Check out the tag for the code, then read the committed results directory for the data.

v2-gap-06-native-datastore-families

embedded documents · gap

No native document, key-value, graph or wide-column database was measured: JSONB/state is not evidence for a native document database, and Redis was measured only as a cache. Cross-family transfers to native datastores are analogy, not evidence.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: do not transfer the embedded-document or cache results to a native family

Source: `studies/05-cache-consistency/reports/20260921T-survey3.md` · run `20260921T-survey3` · tag `run/05-cache-consistency/20260921T-survey3`

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted

Source: `studies/02-ticket-booking/reports/20260920T234953Z.md` · run `20260920T234953Z` · tag `run/02-ticket-booking/20260920T234953Z`

9.2 Reproducing the book itself

The book is built in Podman from a pinned toolchain; the manifest records every input and the resulting hash.

```
book/build.sh          # builds the pinned image, compiles, verifies, writes the
                        manifest
cat book/dist/build-manifest.json
```

9.3 Reproducing a measured run

```
git checkout run/<study>/<run-id>    # the producing code state
infra/<topology>.sh up                # take the benchmark lock first
cd studies/<study> && ./run-<engine>.sh
```

A run's results live in `studies/<study>/results/<run-id>/`; the report beside them is generated, never hand-edited, and its inputs digest identifies the data.

10 Concepts

11 Information placement

CONCEPT · INFORMATION PLACEMENT

Placement is the decision the rest of the book hangs on: which single fact owns which question, and what is allowed to exist only because it answers a question faster. Every design in the corpus is a different answer to “where does this fact live, and who maintains it”.

11.1 The ladder

- **Derive it.** Answer the question from normalized facts at read time. Nothing to maintain, nothing to go stale; the cost is paid per read.
- **Index an existing fact.** Add the structure that turns a scan into a lookup. The cheapest derived object, but it taxes every write and only helps questions the schema can already express.
- **Materialize it.** Store the aggregate on the parent. Big read wins, a maintenance cost on every child write, and a correctness question whenever a write path forgets it.
- **Copy it down (rolldown).** Put parent information on the child to avoid a join. Measured as a package with its indexes, never as a bare column.
- **Copy it out (cache).** Serve the answer from beside the database. The fastest read and the only step that adds a consistency contract to the design.

11.2 The rule that comes before performance

Direct evidence

Ask what the schema must be able to **answer** before asking how fast it answers. Two measured answerability failures make this concrete: refunds were unanswerable in 14 of 15 ticketing designs, and no reserved-seating design could size a hold window because releasing an expired hold erases the evidence that it existed.

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33-35% more storage.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: answerability is a schema fact, not a timing; the +33-35% storage delta is deterministic and stable across three runs; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

11.3 Boundaries

Coverage gap

Placement says nothing about **physical** colocation. The corpus does not verify where a cluster put its data, so a placement claim here is a design claim, not a topology result.

12 Indexes and hot rows

CONCEPT · INDEXES AND HOT ROWS

Two different tools get confused because both look like “make the query fast”. An index changes how many rows the engine touches; a hot-row decision changes who may touch one row at the same time.

12.1 Indexes

An index answers a question the schema can already express, at the cost of every write to the indexed column. In the charity tree the first index was the first large read win, and flattening the foreign key (a rolldown package) bought more still.

12.2 Hot rows

When one row is the contention point — one event’s ticket count, one product’s stock — the design question is arbitration, not structure. Pre-created seats taken by compare-and-set or SKIP LOCKED sold 4–8x faster than any single counter row; the counter also blocks unrelated edits to the same row, which is a correctness-adjacent cost the throughput number hides.

12.3 Direct evidence

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-01-study01-d2d3-copied-key-package](#)

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4-8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: [studies/02-ticket-booking/reports/20260913T021206Z.md](#) · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

12.4 Boundaries

The hot-row result is one workload at small scale on one host. Read it as “the counter row serialises everything”, not as a capacity figure.

13 Concurrency control

CONCEPT · CONCURRENCY CONTROL

Two strategies at minimum: check a version and retry, or take a lock before updating. Both protect the same invariant; they differ in where contention is paid and what the failure looks like.

13.1 Optimistic (version-checked)

The writer reads, checks that the version is unchanged, writes conditionally, and retries on a mismatch. Under low contention it is close to free; under high contention retries become the workload. The measured upper bound put lock-before-update at 1.76x the optimistic design under one hot key and 16 writers, with the caveat that the run could not separate the lock's advantage from extra per-retry harness work.

13.2 Pessimistic (lock-before-update)

The writer waits for the row, then updates. Latency per operation is higher and the waiting is visible, but the retry loop disappears. On a single engine primitive, SKIP LOCKED and compare-and-set both avoid the counter-row bottleneck measured in ticketing.

13.3 The negative control matters here

An unchecked read-modify-write lost 93.7% of acknowledged updates in the same run that measured the lock advantage. A concurrency result without its lost-update control is not evidence.

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

13.4 Boundaries

One key, one engine, one trial, small scale. The ratio is an upper bound and a point, not a curve.

14 Derived state and history

CONCEPT · DERIVED STATE AND HISTORY

A rollup, a copied key and an append-only ledger are all “a fact stored twice”. They differ in what they can answer after the fact and what they cost on every write.

14.1 Maintenance is the design

A rollup can be maintained by the application or by a database trigger. The trigger makes the guarantee structural; the measured cost was 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 to 42 ms), while the application rollup kept writes near the reference for the same read gain. Neither is universally right: choose who must never forget.

14.2 History is answerability

Ledgers and snapshots are not faster; they are the only structures that can answer a question after the event was undone. The refunds window was unanswerable in 14 of 15 ticketing designs and cost about 33–35% more storage with a ledger. That is a correctness purchase, priced honestly.

14.3 Direct evidence

v2-13-trigger-rollup-cost

rollups / materialized aggregates · single_run_directional

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (n4_rollup_app) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92-283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7-5.7x (not the headline's narrower 1.7-2.1x).

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 32 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

14.4 Boundaries

The ledger's write cost is engine- and order-sensitive at the small arm; quote the run's own range and the disagreement, not a headline multiplier. The load-time multiplier is not published at all.

15 Expiry and clock authority

CONCEPT · EXPIRY AND CLOCK AUTHORITY

Expiry is a write that no client issues. That makes it the place where “whose clock is right” stops being philosophical: the sweeper, the application clock and the database all disagree eventually, and the design must say who is allowed to decide.

15.1 What the corpus shows

YugabyteDB refused valid hold confirmations under a guarded design, and retrying the refused statement once removed every refusal at no measurable cost. The practical lesson is smaller than it sounds: an expiry design needs a retry story, because the mechanism can refuse a correct write.

15.2 What it cannot yet show

No design can answer the funnel that would size a hold window. Releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat that was never held. The missing evidence is a schema property, not a missing measurement.

v2-10-guarded-confirm-refusals

normalized facts · single_run_directional

On YugabyteDB a guarded confirmation is not enough: correct designs refused valid holds, and retrying the refused statement once removed every refusal at no measurable cost; on PostgreSQL the guarantee held everywhere.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag

Source: studies/03-reserved-seating/reports/20260915T002411Z.md · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

15.3 Boundaries

Do not transfer a single-engine expiry result to a clock-synchronised cluster without a placement and clock test; the corpus has neither.

16 Cache consistency

CONCEPT · CACHE CONSISTENCY

A cache is a read copy with a freshness contract. Invalidation is a fence — a rule that refuses a publication whose input moved — not a best-effort deletion. This is the concept the external- read-copies family exists to hold.

16.1 The gain

Adding a cache to warm cacheable reads bought 2.2–3.5x in throughput and an order of magnitude in p99 (15.8–27.1 ms to 0.8–2.0 ms). This is an **add-cache** result: the Redis container’s own budget is outside the comparison, so it is not an equal-total-resource efficiency claim.

16.2 The failure mode

Three logical application instances sharing one cache under write-through + relaxed freshness recorded about 87% wrong reads; every single-instance relaxed cell recorded zero. A private in-memory lease cannot coordinate instances, and a database version token does not remove the window by itself. The corrected attribution: 87.81% belongs to the owned model, 87.24% to legacy.

16.3 Freshness is a contract, not a speed knob

Every controlled strict/relaxed pair differed by –14.2% to +12.3% in read throughput — inside the run’s noise. The visible cost sits on the write path, where strictness adds fences. Do not read “strict freshness costs nothing” from the read table.

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2-3.5x in throughput and an order of magnitude in p99 (15.8-27.1 ms to 0.8-2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); no YugabyteDB, cluster, verified placement or open-loop demand

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: `conf-06-claim12-full-noise-range`, `conf-04-study05-db-only-cache`

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: `studies/05-cache-consistency/reports/20260921T-survey3.md` · `run 20260921T-survey3` · `tag run/05-cache-consistency/20260921T-survey3`

16.4 Boundaries

Redis was measured as a cache, never as an authoritative key-value store. JSONB is not a native document store. Transfer either result to a native family only as a labelled analogy.

17 Distributed placement

CONCEPT · DISTRIBUTED PLACEMENT

Node count, replication factor and actual data placement are three separate facts. A 3-node cluster on one laptop has the first two and none of the third’s network consequences.

17.1 What the corpus measures

Studies 01–03 run PostgreSQL single-node and YugabyteDB single-node and 3-node RF=3 cells. The multi-node cells show placement-shaped effects — a section-sharded layout collapsed point lookups to 26 ops/s on YugabyteDB — but the balance of tablets across nodes is unexplained.

17.2 What it does not measure

Physical colocation is unverified, and balanced client access across endpoints is unproven. A single query endpoint stands in for cluster access in every multi-node run. Both are recorded gaps, not null results.

v2-gap-05-colocation-unverified

rolldown / duplicated data · gap

Physical data placement is a common open gap: no study verifies actual node placement. Study 01 D7/D24 and Study 03 L3 are placement intent with measured effects, not verified colocation; Study 02 records its own placement gap; Study 04 y1/y2 and Study 05’s placement pair were never executed.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: containers on one laptop are not evidence of colocation; physical placement is undocumented to today’s standard, not proven uncolocated

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

v2-gap-07-real-network-balanced-endpoints

normalized facts · gap

Every multi-node result runs on one laptop with no real network, and balanced client access across cluster endpoints is not proven. Multi-node ratios are placement experiments, not distributed-systems results.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: no real network between nodes; endpoint distribution across the cluster is not measured

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

v2-gap-06-native-datastore-families

embedded documents · gap

No native document, key-value, graph or wide-column database was measured: JSONB/state is not evidence for a native document database, and Redis was measured only as a cache. Cross-family transfers to native datastores are analogy, not evidence.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: do not transfer the embedded-document or cache results to a native family

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

17.3 Boundaries

Do not read a single-node-versus-cluster ratio as production capacity guidance. The nodes share the host’s cores and have no real network between them.

18 Reading benchmarks honestly

CONCEPT · READING BENCHMARKS

A benchmark answers the question its harness asks. Most wrong conclusions in this corpus came from reading a number as if it answered a different question, not from a bad run.

18.1 Six questions to ask of any number

- **Closed or open loop?** Every race here is closed loop; its tail is a floor, not an SLO.
- **One trial or many?** Spreads within a measurement are not an across-trial error bar.
- **Per-node or equal-total framing?** Single-vs-cluster answers “add machines”, not “same budget”.
- **What was the design order?** In Study 02 the two arms disagree in sign on PostgreSQL, so no ratio is quotable until a repeated-design control is run.
- **What fired the control?** A correctness gate without a fired negative control proves less than it appears to.
- **What is the cell, and what is the run?** A run’s totals and failures belong to the run; a claim’s support cells are a selected subset.

18.2 Worked examples of a number that cannot be quoted

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92-283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7-5.7x (not the headline's narrower 1.7-2.1x).

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 32 inner iterations.

Confounds: [conf-05-study02-buyer-arms-disagree](#)

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: [studies/02-ticket-booking/reports/20260921T183722Z.md](#) · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted

Source: [studies/02-ticket-booking/reports/20260920T234953Z.md](#) · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-06-claim12-full-noise-range](#), [conf-04-study05-db-only-cache](#)

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

18.3 Boundaries

Laptop topologies with shared cores are a controlled comparison, not a capacity model. The environment caveat belongs wherever a conclusion is drawn.

19 Evidence registry (active v2 claims)

The active registry holds 23 claims. This index is rendered from the registry file at build time; the digest above identifies its content.

19.1 normalized facts

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-01-study01-d2d3-copied-key-package](#)

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4-8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: [studies/02-ticket-booking/reports/20260913T021206Z.md](#) · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-10-guarded-confirm-refusals

normalized facts · single_run_directional

On YugabyteDB a guarded confirmation is not enough: correct designs refused valid holds, and retrying the refused statement once removed every refusal at no measurable cost; on PostgreSQL the guarantee held everywhere.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag

Source: [studies/03-reserved-seating/reports/20260915T002411Z.md](#) · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: [studies/04-configuration-portal/reports/20260921T1215Z-small.md](#) · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-gap-03-study04-missing-designs

normalized facts · gap

Eight of the eighteen planned Study 04 configuration-portal designs are not implemented, and cardinality, cadence/churn, repeated randomized trials, equal-total resources, placement and YugabyteDB are unmeasured. These are coverage gaps, not null results.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: no conclusion may read 'the missing designs would not change this'; owned by task-20260922T025912Z-study04-v2-protocol

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-gap-07-real-network-balanced-endpoints

normalized facts · gap

Every multi-node result runs on one laptop with no real network, and balanced client access across cluster endpoints is not proven. Multi-node ratios are placement experiments, not distributed-systems results.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: no real network between nodes; endpoint distribution across the cluster is not measured

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

19.2 rolldown / duplicated data

v2-05-d3-read-advantage-repeated

rolldown / duplicated data · repeated_controlled

D3's (d3_flattened_fk) read advantage over the normalized indexed design survives five fresh loads; the recency SQL and actual heap avoidance explain the gain, while its insert cost remains imprecise.

Trials: 1 whole-run replications, 5 fresh-load trials per cell, 1 inner iterations.

Limits: five fresh-load trials per cell, one session; the insert-cost estimate is imprecise and must not be quoted as a ratio

Source: studies/01-charity-tree/reports/20260913T125342Z-v3.md · run 20260913T125342Z-v3 · tag run/01-charity-tree/20260913T125342Z-v3

v2-gap-05-colocation-unverified

rolldown / duplicated data · gap

Physical data placement is a common open gap: no study verifies actual node placement. Study 01 D7/D24 and Study 03 L3 are placement intent with measured effects, not verified colocation; Study 02 records its own placement gap; Study 04 y1/y2 and Study 05's placement pair were never executed.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: containers on one laptop are not evidence of colocation; physical placement is undocumented to today's standard, not proven uncolocated

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

19.3 rollups / materialized aggregates

v2-02-rollup-family

rollups / materialized aggregates · single_run_directional

Parent rollups win the aggregate read score (D4 rollup/trg 73.8x and D5 rollup/app 69.9x against D1 minimal) and pay for it on inserts (2.6k/s and 2.1k/s against 6.2k/s).

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial per cell; PostgreSQL 1 node numbers shown; YugabyteDB in the same report; the write cost is per-insert on a fresh load

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-04-recency-maintained-index

rollups / materialized aggregates · single_run_directional

The recency flag works, but an index on the rollup column answers the same question faster: comparing `d20_recency_flag` with `d22_recency_rollup_idx` in the same run gives the reported read speedup on PostgreSQL and YugabyteDB.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-02-study01-d20d22-write-axis](#)

Limits: the write comparison is confounded: d22 also maintains D4's multi-column rollup package, so its write cost is not the index alone; three trials per cell; claim-02's v1 trials field was recorded as 1 and is corrected here

Source: [studies/01-charity-tree/reports/20260916T090036Z-v3.md](#) · [run 20260916T090036Z-v3](#) · [tag run/01-charity-tree/20260916T090036Z-v3](#)

v2-13-trigger-rollup-cost

[rollups / materialized aggregates · single_run_directional](#)

Maintaining the portal overview with a per-row trigger (`n3_rollup_trigger`) costs 6.85x on whole-configuration replacement (83.7 vs 573 ops/s, p99 2.5 -> 42 ms) while the application rollup (`n4_rollup_app`) keeps writes near the reference (593 ops/s) for the same ~15x read gain.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: PostgreSQL single node, small scale, one trial; the big effects (6.85x write, ~15x read) are above the run's ~1.7x read floor; smaller differences are not attributable; the trigger's product is a structural guarantee, not speed

Source: [studies/04-configuration-portal/reports/20260921T1215Z-small.md](#) · [run 20260921T1215Z-small](#) · [tag run/04-configuration-portal/20260921T1215Z-small](#)

19.4 embedded documents

v2-03-embedding-not-ahead

[embedded documents · single_run_directional](#)

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: 2.4x D1 minimal's read score, below both indexed designs, with a 0.57x insert rate.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database

Source: [studies/01-charity-tree/reports/20260912-small.md](#) · [run 20260912-small](#) · [tag none \(legacy\)](#)

v2-11-section-seat-map

[embedded documents · single_run_directional](#)

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5-11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-03-study03-layout-family-package](#)

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: [studies/03-reserved-seating/reports/20260921T231328Z.md](#) · [run 20260921T231328Z](#) · [tag run/03-reserved-seating/20260921T231328Z](#)

v2-gap-06-native-datastore-families

[embedded documents · gap](#)

No native document, key-value, graph or wide-column database was measured: JSONB/state is not evidence for a native document database, and Redis was measured only as a cache. Cross-family transfers to native datastores are analogy, not evidence.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: do not transfer the embedded-document or cache results to a native family

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · [run 20260921T-survey3](#) · [tag run/05-cache-consistency/20260921T-survey3](#)

19.5 append-only history / snapshots

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33-35% more storage.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: answerability is a schema fact, not a timing; the +33-35% storage delta is deterministic and stable across three runs; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-08-ledger-race-cost-128-buyers

append-only history / snapshots · single_run_directional

At 128 buyers the ledger costs 1.1-2.3x of the sell-out race on PostgreSQL and 1.8-2.2x on YugabyteDB with tight spreads, 2.1-2.5x on the refund itself, and not one under-booked seat anywhere; it delays the organiser's own edit by up to 5x.

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 128 inner iterations.

Confounds: conf-05-study02-buyer-arms-disagree

Limits: one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier

Source: studies/02-ticket-booking/reports/20260921T205212Z.md · run 20260921T205212Z · tag run/02-ticket-booking/20260921T205212Z

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92-283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7-5.7x (not the headline's narrower 1.7-2.1x).

Trials: 1 whole-run replications, 3 fresh-load trials per cell, 32 inner iterations.

Confounds: conf-05-study02-buyer-arms-disagree

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Limits: a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

19.6 external read copies / caches

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2-3.5x in throughput and an order of magnitude in p99 (15.8-27.1 ms to 0.8-2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); no YugabyteDB, cluster, verified placement or open-loop demand

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. The visible cost is on the write path (more fences), so this must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replications, 1 fresh-load trials per cell, 1 inner iterations.

Confounds: [conf-06-claim12-full-noise-range](#), [conf-04-study05-db-only-cache](#)

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-gap-04-study05-unmeasured-regimes

external read copies / caches · gap

Real-TTL churn, medium scale, repeated trials, cache resource accounting, YugabyteDB/cluster, verified placement and open-loop demand are unmeasured for Study 05.

Trials: 0 whole-run replications, 0 fresh-load trials per cell, 0 inner iterations.

Confounds: [conf-04-study05-db-only-cache](#)

Limits: owned by task-20260922T025912Z-study05-v2-protocol

Source: [studies/05-cache-consistency/reports/20260921T-survey3.md](#) · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

19.7 Retired v1 claims

- [claim-gap-04-study02-03-validation-open](#) — The validation it describes as open was completed and integrated 2026-09-22 (task-20260922T025912Z-study02-03-v2-validation). Not carried into v2.